# INDEX


| PROJECT. NO | PROJECT NAME | PAGE NO |
|---|---|---|
| 1 | Iris Dataset and SVM | 1 |
| 2 | CA Housing Data and MLP NN | 4 |
| 3 | TensorBoard PlayGround | 5 |
| 4 | Batch Normalization with Keras using MNIST | 9 |
| 5 | CNN with Keras using Fashion MNIST | 11 |
| 6 | CNN for Fashion MNIST data with Conv2D and MaxPool2D layers | 13 |
| 7 | CNN Architecture for Fashion MNIST classification with specific layer configurations. | 15 |
| 8 | RNN QuickDraw | 17 |

| Project. No: 1 | Iris Dataset and SVM |
|---|---|

**Project Task:**

1. Load the IRIS data (Make X to be [either setosa or versicolor] or NOT)
2. Find out how many cases are there. Length of X
3. Print first 10 cases of X and Y
4. Find what attributes are there for X
5. Find mean and std of each attribute
6. Use SVC (kernel='linear', …) to train it
7. Test with a few sample data that you create
8. Can you make your SVC classifier to get 95% or better accuracy?

You may use different kernel for your SVC (or Linear).

**Implementation:**

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
iris = datasets.load_iris()
X = iris.data
y = iris.target
mask = y < 2
X, y = X[mask], y[mask]
num_cases = len(X)
print("Number of cases:", num_cases)
print("First 10 cases of X:", X[:10])
print("First 10 cases of Y:", y[:10])
attributes = iris.feature_names
```

```
print("Attributes of X:", attributes)
df = pd.DataFrame(X, columns=attributes)
print("Mean of each attribute:\n", df.mean())
print("Standard deviation of each attribute:\n", df.std())
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
svc_linear = SVC(kernel='linear')
svc_linear.fit(X_train, y_train)
sample_data = scaler.transform([[5.1, 3.5, 1.4, 0.2], [6.0, 2.9, 4.5, 1.5]])  # Example data points
print("Sample data predictions:", svc_linear.predict(sample_data))
y_pred = svc_linear.predict(X_test)
accuracy_linear = accuracy_score(y_test, y_pred)
print("Accuracy with linear kernel:", accuracy_linear)
svc_rbf = SVC(kernel='rbf')
svc_rbf.fit(X_train, y_train)
y_pred_rbf = svc_rbf.predict(X_test)
accuracy_rbf = accuracy_score(y_test, y_pred_rbf)
print("Accuracy with RBF kernel:", accuracy_rbf)
svc_poly = SVC(kernel='poly', degree=3)
svc_poly.fit(X_train, y_train)
y_pred_poly = svc_poly.predict(X_test)
accuracy_poly = accuracy_score(y_test, y_pred_poly)
print("Accuracy with Polynomial kernel:", accuracy_poly)
svc_sigmoid = SVC(kernel='sigmoid')
svc_sigmoid.fit(X_train, y_train)
y_pred_sigmoid = svc_sigmoid.predict(X_test)
accuracy_sigmoid = accuracy_score(y_test, y_pred_sigmoid)
print("Accuracy with Sigmoid kernel:", accuracy_sigmoid)
best_model = svc_rbf if accuracy_rbf > max(accuracy_linear, accuracy_poly, accuracy_sigmoid) else svc_linear
```

```
best_model = svc_poly if accuracy_poly > max(accuracy_rbf, accuracy_linear,
accuracy_sigmoid) else best_model

best_model = svc_sigmoid if accuracy_sigmoid > max(accuracy_rbf, accuracy_poly,
accuracy_linear) else best_model

y_pred_best = best_model.predict(X_test)

conf_matrix = confusion_matrix(y_test, y_pred_best)

disp = ConfusionMatrixDisplay(confusion_matrix=conf_matrix,
display_labels=iris.target_names[:2])

disp.plot()

plt.show()
```

**Output:**

```
First 10 cases of X:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]]

Corresponding Y values:
[0 0 0 0 0 0 0 0 0 0]

Attributes for X:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Mean of each attribute:
sepal length (cm): 5.471000000000003
sepal width (cm): 3.099
petal length (cm): 2.861
petal width (cm): 0.7859999999999999

Standard deviation of each attribute:
sepal length (cm): 0.6384817930058776
sepal width (cm): 0.4763391648814948
petal length (cm): 1.4422825659349836
petal width (cm): 0.5623201934841042

Fitting 5 folds for each of 48 candidates, totalling 240 fits
Best accuracy after tuning: 1.0
```

| Project. No: 2 | CA Housing Data and MLP NN |
|---|---|

**Project Task:**

(1) Fetching and splitting the dataset using fetch_california_housing()

(2) Creating a pipeline to standardize the input features and model with MLPRegressor with 3 hidden_layers with hidden_layer_sizes=[50, 50, 50]

(3) Then the code trains the model and evaluates its validation error using mean squared error

**Implementation:**

```
from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.pipeline import Pipeline

from sklearn.preprocessing import StandardScaler

from sklearn.neural_network import MLPRegressor

from sklearn.metrics import mean_squared_error

housing = fetch_california_housing()

X_train, X_test, y_train, y_test = train_test_split(housing.data, housing.target, test_size=0.2, random_state=42)

mlp_reg = MLPRegressor(hidden_layer_sizes=(50, 50, 50), activation='relu', solver='adam', alpha=0.0001, batch_size='auto', learning_rate='constant', learning_rate_init=0.001, max_iter=1000, shuffle=True, random_state=42, tol=1e-4)

pipeline = Pipeline([('scaler', StandardScaler()),('mlp', mlp_reg)])

pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)

mse = mean_squared_error(y_test, y_pred)

print(f"Mean Squared Error on test set: {mse:.4f}")
```

**Output:**

```
Pipeline
Pipeline(steps=[('scaler', StandardScaler()),
                ('mlp',
                 MLPRegressor(hidden_layer_sizes=(50, 50, 50), max_iter=1000,
                              random_state=42))])
StandardScaler
StandardScaler()
MLPRegressor
MLPRegressor(hidden_layer_sizes=(50, 50, 50), max_iter=1000, random_state=42)

Mean Squared Error on test set: 0.2700
```

| Project. No: 3 | TensorBoard PlayGround |
|---|---|

**Project Task:**

1. Work with TensorBoard PlayGround with 4 example data to create your NN to be trained. bit.ly/Tfplayground

2. Your task is to create and train your NN to get Test error rate less than 0.005 for each data case. There are 4 data cases to be worked out.

**Implementation:**

Data Case 1 – Circle:

- Network Configuration:
  - o Input Layer: 2 neurons
  - o Hidden Layers: [8, 8, 8]
  - o Output Layer: 1 neuron
  - o Activation Function: ReLU
  - o Learning Rate: 0.01
  - o Regularization: None
  - o Ratio of Training to Test Data: 0.5
- Expected Training Details:
  - o Epochs: 551
  - o Training Loss: 0.0
  - o Test Loss: 0.001

Data Case 2 – Exclusive OR:

- Network Configuration:
  - o Input Layer: 2 neurons
  - o Hidden Layers: [8, 8, 8]
  - o Output Layer: 1 neuron
  - o Activation Function: Tanh
  - o Learning Rate: 0.03
  - o Regularization: None
  - o Ratio of Training to Test Data: 0.5
- Training Details:

- o Epochs: 266
- o Training Loss: 0.0
- o Test Loss: 0.002

Data Case 3 – Gaussian:

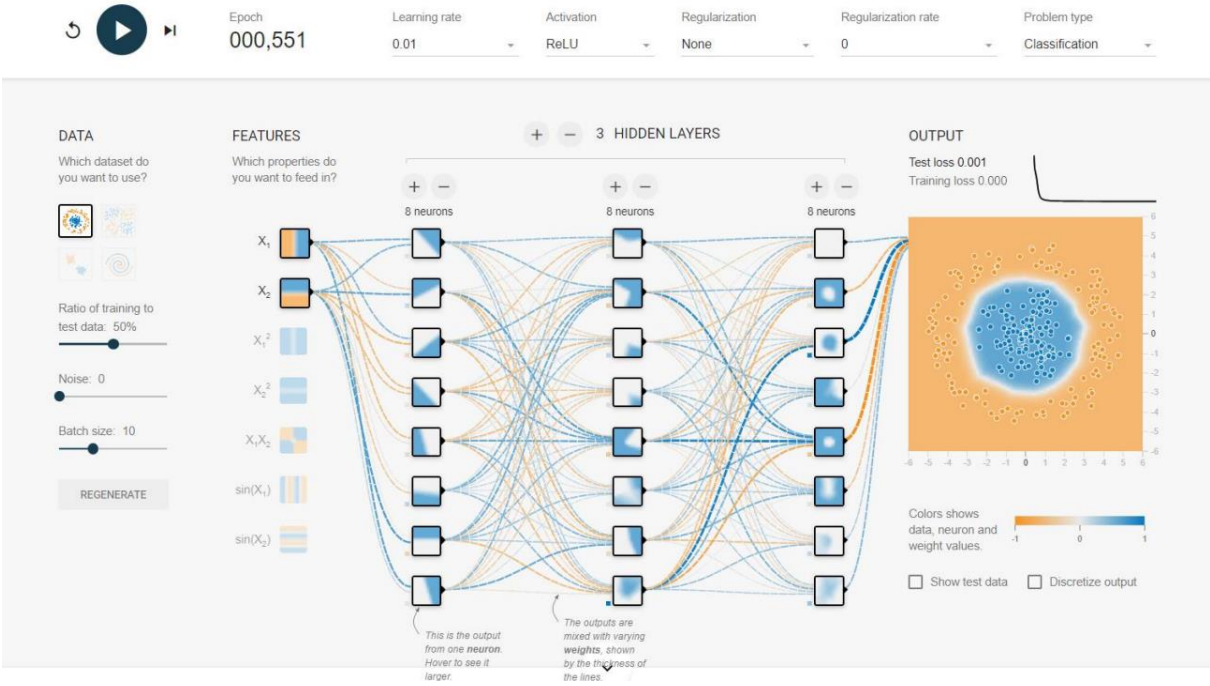
- Network Configuration:
  - o Input Layer: 2 neurons
  - o Hidden Layers: [4, 4]
  - o Output Layer: 1 neuron
  - o Activation Function: ReLU
  - o Learning Rate: 0.03
  - o Regularization: None
  - o Ratio of Training to Test Data: 0.5
- Training Details:
  - o Epochs: 116
  - o Training Loss: 0.0
  - o Test Loss: 0.001

Data Case 4 – Spiral:

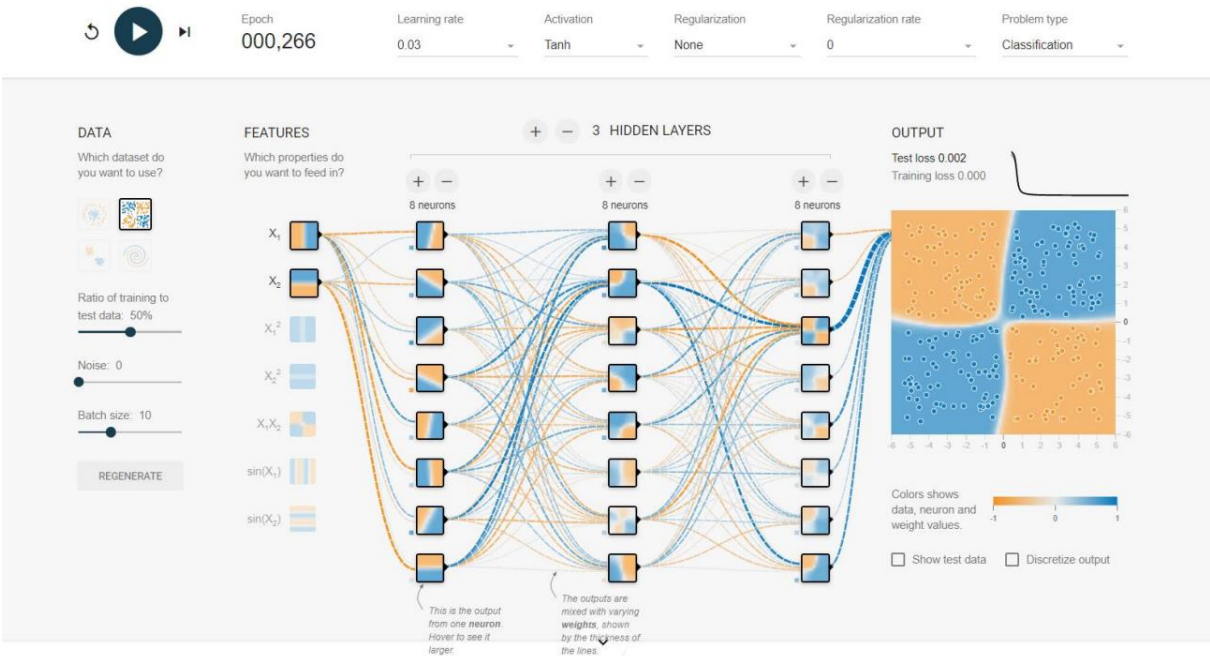
- Network Configuration:
  - o Input Layer: 2 neurons
  - o Hidden Layers: [8, 8, 8, 8]
  - o Output Layer: 1 neuron
  - o Activation Function: Tanh
  - o Learning Rate: 0.01
  - o Regularization: None
  - o Ratio of Training to Test Data: 0.5
- Training Details:
  - o Epochs: 44
  - o Training Loss: 0.0
  - o Test Loss: 0.0

## Output:

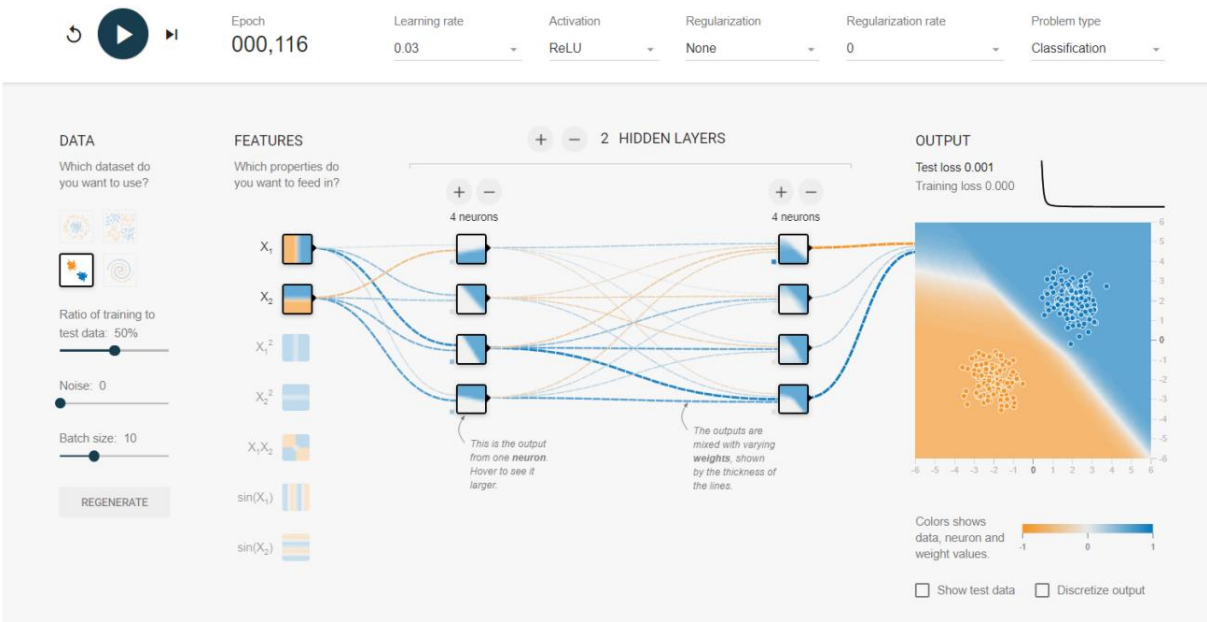
## Circle:



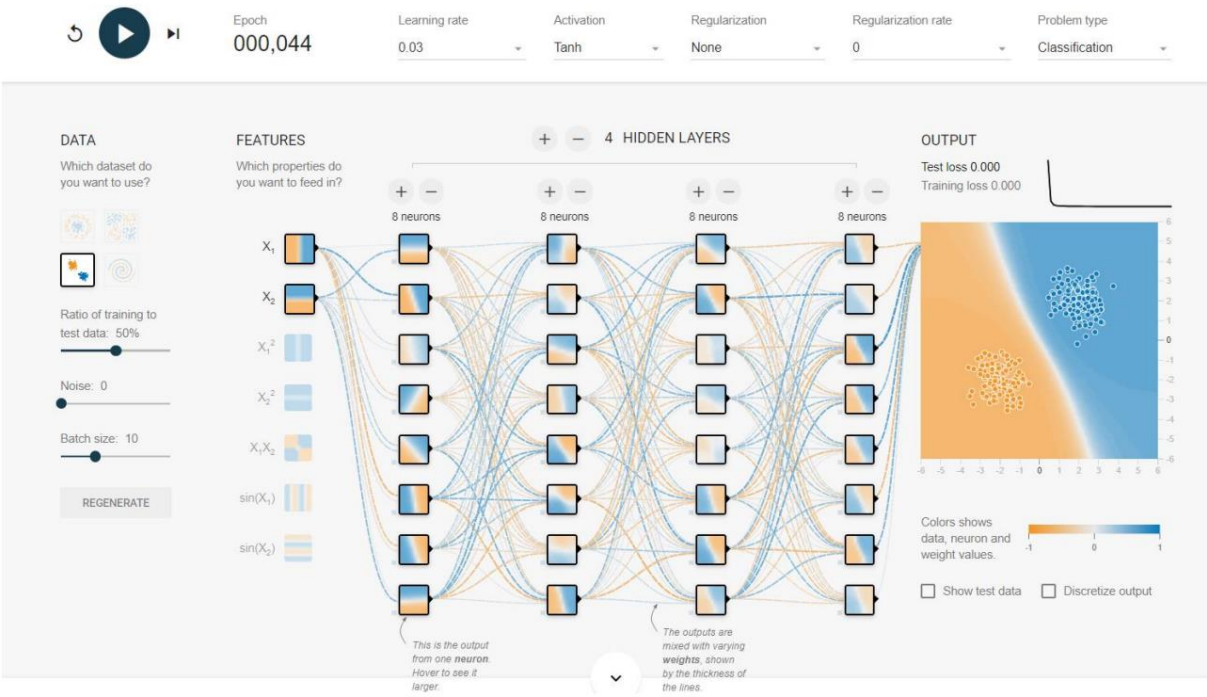

## Exclusive OR:



**Gaussian:**




**Spiral:**



| Project. No: 4 | Batch Normalization with Keras using MNIST |
|---|---|

**Project Task:**

1. Load data.

2. Design and Create Model with input layer (to be flattened), two hidden layers (with 300 and 100 neurons with relu activation functions and kernel initializer of he_normal, and output layer of 10 neurons with activation function of softmax.

3. Train your model and evaluate it.

4. Extend your model to have 3 hidden layers (instead of 2).

5. Can you achieve accuracy (> 95%)?

**Implementation:**

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.datasets import mnist

from tensorflow.keras.utils import to_categorical

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, BatchNormalization, Activation, Input

from tensorflow.keras.initializers import HeNormal

(X_train, y_train), (X_test, y_test) = mnist.load_data()

X_train = X_train.astype('float32') / 255.0

X_test = X_test.astype('float32') / 255.0

y_train = to_categorical(y_train, 10)

y_test = to_categorical(y_test, 10)

X_train = X_train.reshape(-1, 28*28)

X_test = X_test.reshape(-1, 28*28)

initializer = HeNormal()

model = Sequential()

# Input Layer

model.add(Input(shape=(28*28,)))

model.add(Dense(300, kernel_initializer=initializer))

model.add(BatchNormalization())

model.add(Activation('relu'))
```

```
model.add(Dense(100, kernel_initializer=initializer))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dense(10, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2,
verbose=2)
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=2)
print(f'Test accuracy: {test_acc * 100:.2f}%')
# Create the extended model
extended_model = Sequential()
extended_model.add(Input(shape=(28*28,)))
extended_model.add(Dense(300, kernel_initializer=initializer))
extended_model.add(BatchNormalization())
extended_model.add(Activation('relu'))
extended_model.add(Dense(100, kernel_initializer=initializer))
extended_model.add(BatchNormalization())
extended_model.add(Activation('relu'))
extended_model.add(Dense(100, kernel_initializer=initializer))
extended_model.add(BatchNormalization())
extended_model.add(Activation('relu'))
extended_model.add(Dense(10, activation='softmax'))
extended_model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
extended_history = extended_model.fit(X_train, y_train, epochs=20, batch_size=32,
validation_split=0.2, verbose=2)
extended_test_loss, extended_test_acc = extended_model.evaluate(X_test, y_test, verbose=2)
print(f'Extended Test accuracy: {extended_test_acc * 100:.2f}%')
```

**Output:**

**Accuracy for 2 layers:**

```
313/313 - 2s - 6ms/step - accuracy: 0.9822 - loss: 0.0720
Test accuracy: 98.22%
```

**Accuracy for 3 layers:**

```
313/313 - 1s - 4ms/step - accuracy: 0.9828 - loss: 0.0679
Extended Test accuracy: 98.28%
```

| Project. No: 5 | CNN with Keras using Fashion MNIST |
|---|---|

**Project Task:**

1. Load the data (images of 28x28). Add normalization layer. The class names of images are: class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]

2. Design and create a neural net for Fashion MNIST with 100 hidden layers (each with 100 neurons), using the SELU activation function, loss with cross-entropy, learning rate with 0.001, accuracy for metrics. Note that your model should use random seed of 123 for your model to produce the same result each time.

3. Train your model and evaluate it for accuracy, to achieve accuracy of over-99% (accuracy > 99%). You may try more epoch or other options for better result to be achieved.

**Implementation:**

```
import numpy as np

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers, models

np.random.seed(123)

tf.random.set_seed(123)

fashion_mnist = keras.datasets.fashion_mnist

(train_images, train_labels), (test_images, test_labels)=fashion_mnist.load_data()

train_images = train_images/255.0

test_images=test_images/255.0

class_names=["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "shirt",
Sneaker", "Bag", "Ankle boot"]

model=models.Sequential()

model.add(layers.Input (shape=(28, 28)))

model.add(layers.Reshape((28, 28, 1)))

model.add(layers.Conv2D(32, (3, 3), activation='selu'))

model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(64, (3, 3), activation='selu'))

model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(64, (3, 3), activation='selu'))
```

```
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(64, (3, 3), activation='selu'))

model.add(layers.Flatten())

for i in range(100):

        model.add(layers.Dense(100, activation='selu'))

model.add(layers.Dense(10, activation='softmax'))

model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history=model.fit(train_images, train_labels, epochs=18, validation_data=(test_images,
test_labels), verbose=1)

test_loss, test_acc=model.evaluate(test_images, test_labels, verbose=2)

print(f'Test accuracy: {test_acc}')

model.summary()
```

**Output:**

```
313/313 - 2s - loss: 2.3218 - accuracy: 0.1000 - 2s/epoch - 7ms/step
Test accuracy: 0.10000000149011612
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 reshape (Reshape)           (None, 28, 28, 1)         0

 conv2d (Conv2D)             (None, 26, 26, 32)        320
```

| Project. No: 6 | CNN for Fashion MNIST data with Conv2D and MaxPool2D layers |
|---|---|

**Project Task:**

1. Load the data to use the first 50000 images for training and the rest of the images for Validation.

2. Your CNN should have 3 layers of (m Conv2D + 1 MaxPool2D) x 4.

3. Compile the model and train the model with 10 epoch with loss="sparse_categorical_crossentropy", optimizer="nadam", metrics=["accuracy"]
What is the accuracy and error?

4. Test your model with first 20 examples in test data. What is the accuracy?

**Implementation:**

```
import tensorflow as tf

tf.random.set_seed(123)

mnist = tf.keras.datasets.fashion_mnist

(X_train_full, y_train_full), (X_test, y_test) = mnist.load_data()

X_train_full = X_train_full / 255.0

X_test = X_test / 255.0

X_train, X_val = X_train_full[:50000], X_train_full[50000:]

y_train, y_val = y_train_full[:50000], y_train_full[50000:]

X_train = X_train[..., tf.newaxis]

X_val = X_val[..., tf.newaxis]

X_test = X_test[..., tf.newaxis]

from tensorflow.keras import layers, models

model = models.Sequential([layers.Input(shape=(28, 28, 1)),layers.Conv2D(64,kernel_size=7,
padding='same', activation='relu',kernel_initializer='he_normal'),
layers.MaxPooling2D(pool_size=2), layers.Conv2D(128,kernel_size=3, padding='same',
activation='relu', kernel_initializer='he_normal'),layers.Conv2D(128, kernel_size=3,
padding='same',activation='relu',kernel_initializer='he_normal'),
layers.MaxPooling2D(pool_size=2), layers.Conv2D(256, kernel_size=3, padding='same',
activation='relu', kernel_initializer='he_normal'), layers.Conv2D(256, kernel_size=3,
padding='same', activation='relu', kernel_initializer='he_normal'),
layers.MaxPooling2D(pool_size=2), layers.Flatten(), layers.Dense(128, activation='relu',
kernel_initializer='he_normal'), layers.Dense(64, activation='relu',
kernel_initializer='he_normal'), layers.Dropout(0.5), layers.Dense(10, activation='softmax') ])

model.compile(optimizer='nadam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=10, validation_data=(X_val, y_val))
```

```
test_loss_full, test_accuracy_full = model.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy on the full test set: {test_accuracy_full * 100:.2f}%")
print(f"Test loss on the full test set: {test_loss_full:.4f}")
X_test_subset = X_test[:20]
y_test_subset = y_test[:20]
test_loss, test_accuracy = model.evaluate(X_test_subset, y_test_subset, verbose=0)
print(f"Test accuracy on the first 20 examples: {test_accuracy * 100:.2f}%")
```

**Output:**

**Full Evaluation:**

```
Test accuracy on the full test set: 90.07%
Test loss on the full test set: 0.3831
```

**First 20 Evaluation:**

```
Test accuracy on the first 20 examples: 95.00%
```

| Project. No: 7 | CNN Architecture for Fashion MNIST classification with specific layer configurations. |
|---|---|

**Project Task:**

1. Use the first 50000 for training and the rest for testing.
2. Your CNN will have the following layers:
    1. Conv2D of 32 filters, Kernel size of (3,3), for input (of 28x28 image)
    2. Conv2D of 64 filters, Kernel size of (3,3), for input (of 28x28 image)
    3. MaxPool with pool size (2,2)
    4. Dropout (with 0.25)
    5. Flatten (28x28 to 1-dim array)
    6. Dense NN (128 neurouns)
    7. Dropout (with 0.5)
    8. Output (for 10 classes)
3. Use activation (relu) for all layers except (softmax).
4. Can you achieve Accuracy of 98%? How many epoch did you use for your training?

**Implementation:**

```
import keras
from keras.datasets import mnist
(X_train, y_train), (X_valid, y_valid) = mnist.load_data()
X_train = X_train.reshape(60000, 28, 28, 1).astype('float32')
X_valid = X_valid.reshape(10000, 28, 28, 1).astype('float32')
X_train /= 255
X_valid /= 255
n_classes = 10
y_train = keras.utils.to_categorical(y_train, n_classes)
y_valid = keras.utils.to_categorical(y_valid, n_classes)
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPooling2D
model = Sequential()
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)))
```

```
model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(n_classes, activation='softmax'))
test_loss, test_accuracy = model.evaluate(X_valid, y_valid, verbose=0)
print("Test loss: {:.2f}%".format(test_loss * 100))
print("Test Accuracy: {:.2f}%".format(test_accuracy * 100))
```

**Output:**

```
Test loss: 2.75%
Test accuracy: 99.21%
```

| Project. No: 8 | RNN QuickDraw |
|---|---|

**Project Task:**

1. Create your own notebook for this exercise (using the sample code provided).
2. Design and implement Classification Model for the SketchRNN dataset
3. Test the model with 5 sample data (for your test)

**Implementation:**

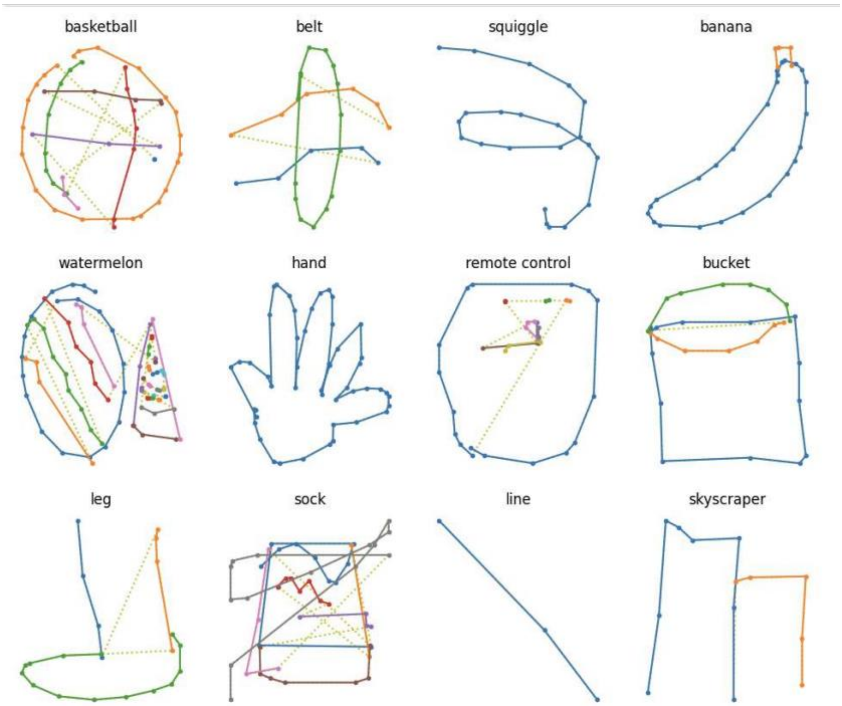
```
import tensorflow as tf
tf_download_root = "http://download.tensorflow.org/data/"
filename = "quickdraw_tutorial_dataset_v1.tar.gz"
filepath = tf.keras.utils.get_file(filename,tf_download_root+filename, cache_dir=".", extract=True)
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
quickdraw_dir = Path(filepath).parent
train_files = sorted([str(path) for path in quickdraw_dir.glob("training.tfrecord-*")])
eval_files = sorted([str(path) for path in quickdraw_dir.glob("eval.tfrecord-*")])
with open(quickdraw_dir / "eval.tfrecord.classes") as test_classes_file:
        test_classes = test_classes_file.readlines()
with open(quickdraw_dir / "training.tfrecord.classes") as train_classes_file:
        train_classes = train_classes_file.readlines()
assert train_classes == test_classes
class_names = [name.strip().lower() for name in train_classes]
sorted(class_names)
def parse(data_batch):
        feature_descriptions = {"ink": tf.io.VarLenFeature(dtype=tf.float32),
                                "shape": tf.io.FixedLenFeature([2], dtype=tf.int64),
                                "class_index": tf.io.FixedLenFeature([1], dtype=tf.int64)
                               }
```

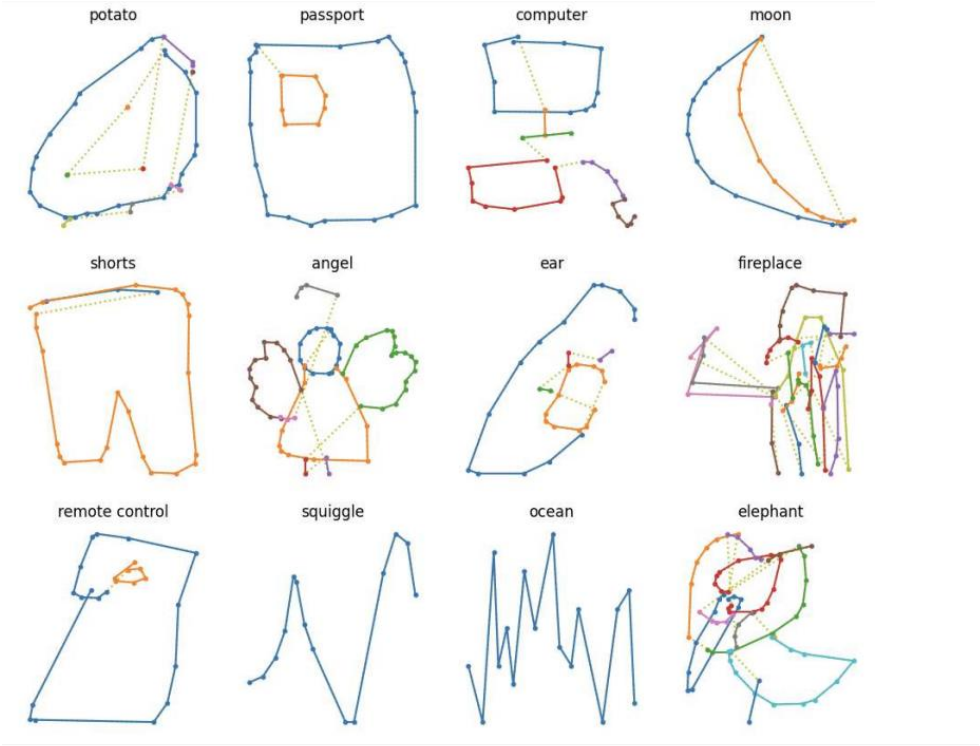
```
    examples = tf.io.parse_example(data_batch, feature_descriptions)
    flat_sketches = tf.sparse.to_dense(examples["ink"])
    sketches = tf.reshape(flat_sketches, shape=[tf.size(data_batch), -1, 3])
    lengths = examples["shape"][:, 0]
    labels = examples["class_index"][:, 0]
    return sketches, lengths, labels
def quickdraw_dataset(filepaths, batch_size=32, shuffle_buffer_size=None,
n_parse_threads=5, n_read_threads=5, cache=False):
    dataset = tf.data.TFRecordDataset(filepaths,num_parallel_reads=n_read_threads)
    if cache:
        dataset = dataset.cache()
    if shuffle_buffer_size:
        dataset = dataset.shuffle(shuffle_buffer_size)
    dataset = dataset.batch(batch_size)
    dataset = dataset.map(parse, num_parallel_calls=n_parse_threads)
    return dataset.prefetch(1)
train_set = quickdraw_dataset(train_files, shuffle_buffer_size=10000)
valid_set = quickdraw_dataset(eval_files[:5])
test_set = quickdraw_dataset(eval_files[5:])
for sketches, lengths, labels in train_set.take(1):
    print("sketches =", sketches)
    print("lengths =", lengths)
    print("labels =", labels)
def draw_sketch(sketch, label=None):
    origin = np.array([[0., 0., 0.]])
    sketch = np.r_[origin, sketch]
    stroke_end_indices = np.argwhere(sketch[:, -1]==1.)[:, 0]
    coordinates = sketch[:, :2].cumsum(axis=0)
    strokes = np.split(coordinates, stroke_end_indices + 1)
    title = class_names[label.numpy()] if label is not None else "Try to guess"
    plt.title(title)
```

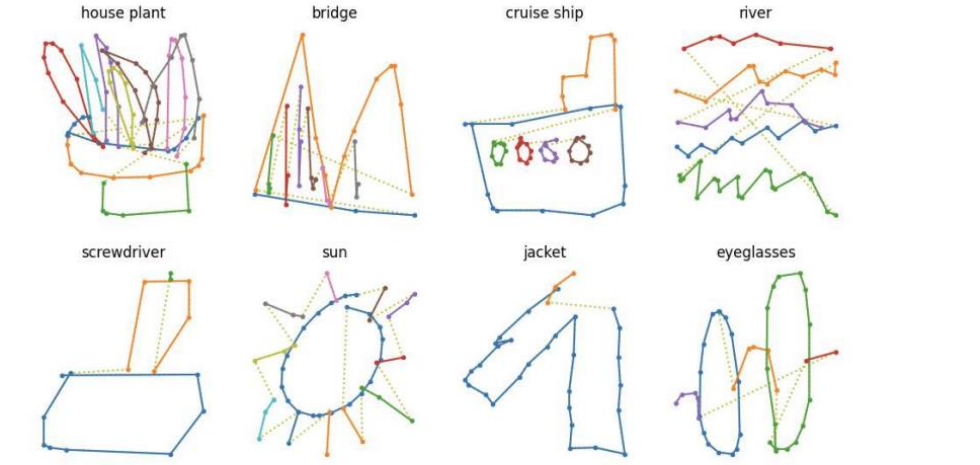
```
    plt.plot(coordinates[:, 0], -coordinates[:, 1], "y:")
    for stroke in strokes:
        plt.plot(stroke[:, 0], -stroke[:, 1], ".-")
    plt.axis("off")
def draw_sketches(sketches, lengths, labels):
    n_sketches = len(sketches)
    n_cols = 4
    n_rows = (n_sketches - 1) // n_cols + 1
    plt.figure(figsize=(n_cols * 3, n_rows * 3.5))
    for index, sketch, length, label in zip(range(n_sketches), sketches, lengths, labels):
        plt.subplot(n_rows, n_cols, index + 1)
        draw_sketch(sketch[:length], label)
    plt.show()
    for sketches, lengths, labels in train_set.take(1):
        draw_sketches(sketches, lengths, labels)
lengths = np.concatenate([lengths for _, lengths, _ in train_set.take(1000)])
plt.hist(lengths, bins=150, density=True)
plt.axis([0, 200, 0, 0.03])
plt.xlabel("length")
plt.ylabel("density")
plt.show()
def crop_long_sketches(dataset, max_length=100):
    return dataset.map(lambda inks, lengths, labels: (inks[:, :max_length], labels))
cropped_train_set = crop_long_sketches(train_set)
cropped_valid_set = crop_long_sketches(valid_set)
cropped_test_set = crop_long_sketches(test_set)
model = tf.keras.Sequential([tf.keras.layers.Conv1D(32, kernel_size=5, strides=2,
activation="relu"), tf.keras.layers.BatchNormalization(), tf.keras.layers.Conv1D(64,
kernel_size=5, strides=2, activation="relu"), tf.keras.layers.BatchNormalization(),
tf.keras.layers.Conv1D(128, kernel_size=3, strides=2, activation="relu"),
tf.keras.layers.BatchNormalization(), tf.keras.layers.LSTM(128, return_sequences=True),
tf.keras.layers.LSTM(128), tf.keras.layers.Dense(len(class_names), activation="softmax")])
optimizer = tf.keras.optimizers.SGD(learning_rate=1e-2, clipnorm=1.)
```

```
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
metrics=["accuracy", "sparse_top_k_categorical_accuracy"])

history = model.fit(cropped_train_set, epochs=2, validation_data=cropped_valid_set)

y_test = np.concatenate([labels for _, _, labels in test_set])

y_probas = model.predict(test_set)

np.mean(tf.keras.metrics.sparse_top_k_categorical_accuracy(y_test, y_probas))

n_new = 10

Y_probas = model.predict(sketches)

top_k = tf.nn.top_k(Y_probas, k=5)

for index in range(n_new):
    plt.figure(figsize=(3, 3.5))
    draw_sketch(sketches[index])
    plt.show()
    print("Top-5 predictions:".format(index + 1))
    for k in range(5):
        class_name = class_names[top_k.indices[index, k]]
        proba = 100 * top_k.values[index, k]
        print(" {}. {} {:.3f}%".format(k + 1, class_name, proba))
    print("Answer: {}".format(class_names[labels[index].numpy()]))
```

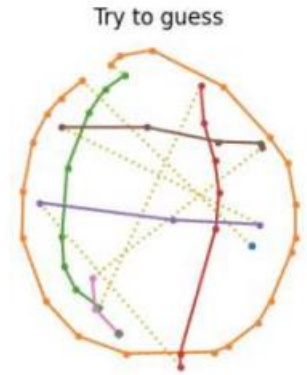
**Output:**



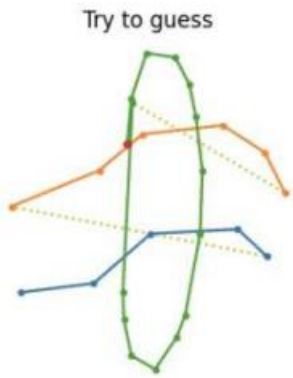


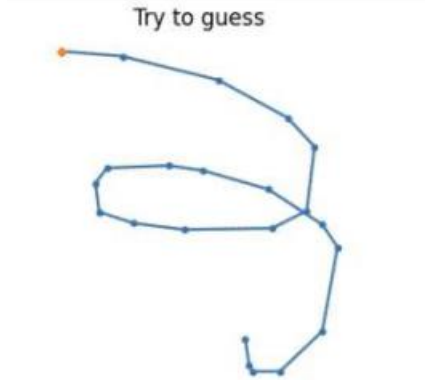

```
32325/32325 [==============================] - 1997s 62ms/step - loss: 5.1678 - accuracy: 0.0403 - sparse_top_k_categorical_accuracy: 0.1301 - val_loss:
4.4608 - val_accuracy: 0.1009 - val_sparse_top_k_categorical_accuracy: 0.2812
5392/5392 [==============================] - 149s 27ms/step
1/1 [==============================] - 1s 945ms/step
```



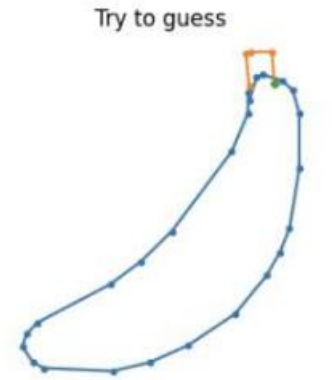

```
Top-5 predictions:
  1. basketball 9.821%
  2. onion 4.883%
  3. tennis racquet 4.615%
  4. baseball 4.122%
  5. watermelon 3.986%
Answer: basketball
```



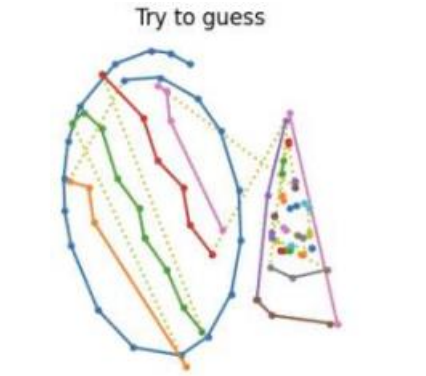

```
Top-5 predictions:
  1. shorts 2.565%
  2. pants 2.519%
  3. ice cream 2.307%
  4. hourglass 2.234%
  5. lipstick 2.188%
Answer: belt
```



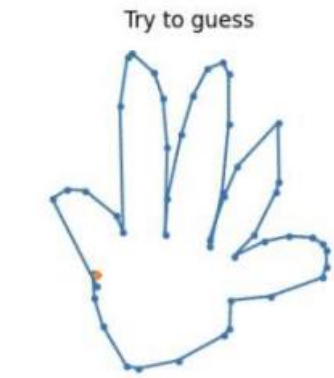

```
Top-5 predictions:
  1. paper clip 5.669%
  2. sock 3.088%
  3. boomerang 2.503%
  4. ear 2.307%
  5. shoe 2.251%
Answer: squiggle
```



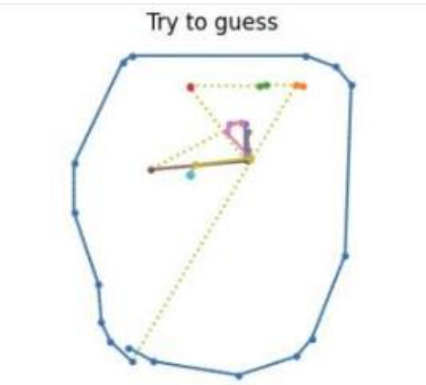

```
Top-5 predictions:
  1. circle 7.783%
  2. potato 5.576%
  3. pear 4.990%
  4. peanut 4.940%
  5. moon 4.839%
Answer: banana
```



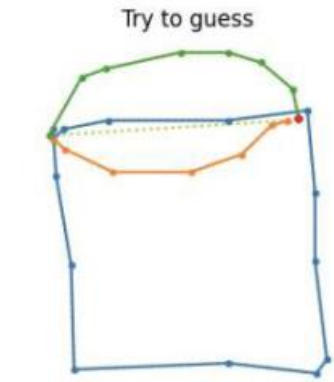

```
Top-5 predictions:
  1. tennis racquet 2.663%
  2. tiger 2.457%
  3. hedgehog 2.414%
  4. brain 2.299%
  5. zebra 2.266%
Answer: watermelon
```




```
Top-5 predictions:
  1. hand 7.578%
  2. tornado 3.393%
  3. tree 3.310%
  4. saw 3.118%
  5. squiggle 2.862%
Answer: hand
```




```
Top-5 predictions:
  1. cake 3.804%
  2. remote control 3.655%
  3. birthday cake 3.356%
  4. map 2.295%
  5. television 2.091%
Answer: remote control
```




```
Top-5 predictions:
  1. picture frame 6.694%
  2. envelope 5.219%
  3. microwave 4.320%
  4. oven 3.808%
  5. television 3.403%
Answer: bucket
```